

IF2211 - K01

TUGAS KECIL 3 STRATEGI ALGORITMA

Penyelesaian Permainan Ice Sliding Puzzle dengan Menggunakan Algoritma Pathfinding



Disusun oleh:

Raymond Jonathan Dwi Putra Julianto 13524059

**SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG**

2026

Contents

1	Permasalahan	2
2	Pendekatan Solusi	2
2.1	Konsep Algoritma	3
2.1.1	Tahap 1: Pembacaan Board	3
2.1.2	Tahap 2: Pembentukan Graph	4
2.1.3	Tahap 3: Pencarian Solusi	5
2.2	Heuristic	6
3	Analisis Algoritma	7
3.1	Definisi State, $g(n)$, dan $f(n)$	7
3.2	Admissibility dan Konsistensi Heuristic	8
3.3	Perbandingan Sifat Formal Algoritma	8
4	Implementasi	10
4.1	Source Program	10
4.2	Struktur Data	11
4.3	Pembacaan Input	12
4.4	Simulasi Sliding	12
4.5	Pembuatan Graph	13
4.6	Pencarian Solusi	13
4.7	Antarmuka CLI dan GUI	14
4.8	Modul IO	14
4.9	Penyimpanan Hasil	14
5	Pengujian	15
5.1	Kasus Uji 1: <code>test1.txt</code> – UCS	15
5.2	Kasus Uji 2: <code>test2.txt</code> – GBFS H1	16
5.3	Kasus Uji 3: <code>test2.txt</code> – GBFS H2	17
5.4	Kasus Uji 4: <code>test3.txt</code> – A* H1	18
5.5	Kasus Uji 5: <code>test3.txt</code> – A* H3	19
5.6	Analisis Hasil Percobaan	19
6	Kesimpulan dan Saran	20
6.1	Kesimpulan	20
6.2	Saran	20
7	Daftar Pustaka	21
8	Lampiran	22

1 Permasalahan

Ice Sliding Puzzle adalah permainan pencarian jalur pada papan dua dimensi. Pemain atau aktor berada pada satu titik awal dan harus mencapai titik tujuan. Berbeda dengan pergerakan biasa, aktor tidak bergerak satu petak lalu berhenti. Ketika aktor memilih arah atas, kanan, bawah, atau kiri, aktor akan terus meluncur pada arah tersebut sampai berhenti tepat sebelum rintangan.

Pada papan permainan terdapat beberapa jenis tile. Tile Z menyatakan posisi awal aktor, tile 0 menyatakan posisi tujuan, tile X menyatakan rintangan, tile L menyatakan lava, tile * menyatakan jalan biasa, dan tile angka 0 sampai 9 menyatakan tile penting. Tile angka harus dilewati sesuai urutan dari angka terkecil. Apabila aktor melewati angka yang lebih besar sebelum waktunya, jalur tersebut dianggap tidak valid. Apabila aktor melewati lava atau bergerak keluar papan tanpa dapat berhenti, jalur tersebut juga dianggap tidak valid.

2 Pendekatan Solusi

Permasalahan ini diselesaikan dengan pendekatan *graph search*. Papan permainan terlebih dahulu dibaca sebagai objek **Board**. Setelah itu, program membentuk graph sederhana. Node pada graph bukan semua tile di papan, melainkan hanya posisi yang dapat menjadi tempat aktor berhenti setelah melakukan sliding. Dengan demikian, satu edge pada graph mewakili satu gerakan sliding lengkap dari satu posisi berhenti ke posisi berhenti lain.

Setiap edge menyimpan arah gerak, node tujuan, cost total tile yang dilalui, dan daftar tile penting yang dilewati selama sliding. Pendekatan ini penting karena tile angka atau finish dapat berada di tengah lintasan sliding. Apabila tile penting berada di tengah lintasan tetapi aktor tidak berhenti di sana, tile tersebut tidak menjadi node tujuan, tetapi tetap dicatat sebagai tile yang dilewati pada edge. Informasi ini kemudian digunakan saat pathfinding untuk memvalidasi urutan angka.

Sebagai contoh, misalkan terdapat potongan papan sebagai berikut. Simbol Z adalah posisi awal, 0 adalah finish, 0 adalah tile penting, dan X adalah rintangan.

```
1 XXXXXX
2 X0***0X
3 X**X**X
4 X****XX
5 XZ***XX
6 XXXXXX
```

Listing 1: Contoh Board Sederhana

Pada representasi board biasa, setiap karakter pada matriks adalah tile. Namun pada graph sederhana, tidak semua tile menjadi node. Pembuatan Node hanya dibuat pada posisi yang dapat menjadi tempat aktor berhenti. Dari posisi awal Z, aktor dapat melakukan sliding ke atas sampai berhenti tepat sebelum rintangan atau melakukan sliding ke kanan sampai berhenti sebelum rintangan. Tile yang hanya dilewati di tengah lintasan tidak otomatis menjadi node.

Sebagai ilustrasi, beberapa node yang terbentuk dapat direpresentasikan seperti berikut.

Node	Posisi	Tile	Alasan Menjadi Node
N0	Baris 4, Kolom 1	Z	Posisi awal aktor.
N1	Baris 1, Kolom 1	0	Hasil slide ke atas dari Z dan berhenti sebelum dinding atas.
N2	Baris 4, Kolom 4	*	Hasil slide ke kanan dari Z dan berhenti sebelum rintangan.
N3	Baris 1, Kolom 5	0	Posisi finish yang dapat menjadi posisi berhenti.

Dari node-node tersebut, edge menyimpan satu gerakan sliding penuh. Contohnya, apabila dari Z aktor bergerak ke atas, maka edge yang terbentuk adalah $N0 \rightarrow N1$ dengan arah U. Apabila dari $N1$ aktor bergerak ke kanan dan melewati beberapa tile kosong sampai berhenti di 0, maka edge yang terbentuk adalah $N1 \rightarrow N3$ dengan arah R. Jika di tengah edge terdapat angka yang dilewati, angka tersebut disimpan pada `passedImportant`, bukan dipaksa menjadi node baru kecuali aktor memang berhenti tepat di tile tersebut.

2.1 Konsep Algoritma

Secara umum, proses pencarian solusi dibagi menjadi tiga tahap utama. Tahap pertama adalah membaca dan memvalidasi input. Tahap kedua adalah membangun graph sederhana dari papan. Tahap ketiga adalah menjalankan algoritma pencarian pada graph tersebut.

2.1.1 Tahap 1: Pembacaan Board

Tahap pertama dilakukan oleh fungsi `readBoardFromFile`. Fungsi ini menerima sebuah path file teks dan mengembalikan objek `Board` yang sudah berisi data tile beserta cost. Pembacaan dilakukan dalam urutan sebagai berikut.

1. Membaca dua bilangan bulat pertama yang menyatakan jumlah baris (R) dan jumlah kolom (C) papan. Apabila salah satu nilai tidak valid atau bukan bilangan positif, program melempar error.
2. Membaca R baris berikutnya sebagai matriks tile. Setiap baris harus memiliki panjang tepat C karakter. Setiap karakter harus berasal dari himpunan tile yang valid, yaitu *, X, L, Z, 0, atau angka 0–9.
3. Selama pembacaan tile, program menghitung jumlah Z dan 0 serta menyimpan angka tile penting yang muncul. Posisi Z disimpan sebagai start dan posisi 0 disimpan sebagai goal.
4. Setelah seluruh tile dibaca, program membaca matriks cost berukuran $R \times C$. Setiap nilai cost wajib berupa bilangan bulat tidak negatif.
5. Setelah seluruh data dibaca, program melakukan validasi akhir. Jumlah Z harus tepat satu, jumlah 0 harus tepat satu, dan angka tile penting harus berurutan mulai dari 0 hingga maksimum yang ditemukan tanpa ada yang loncat. Jumlah angka penting kemudian disimpan sebagai `importantCount`.

Pembacaan dilakukan secara linier sehingga kompleksitasnya adalah $O(R \times C)$, sama dengan ukuran input. Karena validasi dilakukan saat baris dan cost diparse, kesalahan format dapat dideteksi sedini mungkin.

```
1 FUNCTION ReadBoardFromFile(filePath) -> board
2 ALGORITMA
3   input <- buka file pada filePath
4   IF input gagal dibuka THEN
5     throw error "file tidak dapat dibuka"
6
7   baca rowCount, colCount dari input
8   IF rowCount <= 0 OR colCount <= 0 THEN
9     throw error "ukuran peta tidak valid"
10
11   board <- Board(rowCount, colCount)
12   startCount <- 0
13   goalCount <- 0
```

```

14 foundNumbers <- array boolean berukuran 10, semuanya false
15 maxNumber <- -1
16
17 FOR row dari 0 TO rowCount - 1 DO
18   baca line dari input
19   IF panjang(line) != colCount THEN
20     throw error "panjang baris tidak sesuai"
21
22   FOR col dari 0 TO colCount - 1 DO
23     type <- line[col]
24     IF type bukan tile yang valid THEN
25       throw error "karakter peta tidak valid"
26
27     IF type == 'Z' THEN
28       startCount <- startCount + 1
29     ELSE IF type == 'O' THEN
30       goalCount <- goalCount + 1
31     ELSE IF type adalah angka THEN
32       foundNumbers[type - '0'] <- true
33       maxNumber <- max(maxNumber, type - '0')
34
35     board.setType(row, col, type)
36
37   FOR row dari 0 TO rowCount - 1 DO
38     FOR col dari 0 TO colCount - 1 DO
39       baca cost dari input
40       IF cost < 0 THEN
41         throw error "cost peta tidak valid"
42       board.setCost(row, col, cost)
43
44   IF startCount != 1 THEN
45     throw error "harus tepat satu Z"
46   IF goalCount != 1 THEN
47     throw error "harus tepat satu O"
48
49   FOR number dari 0 TO maxNumber DO
50     IF foundNumbers[number] == false THEN
51       throw error "angka harus berurutan mulai dari 0"
52
53   board.importantCount <- maxNumber + 1
54   RETURN board

```

Listing 2: Pseudocode Pembacaan dan Validasi Board

2.1.2 Tahap 2: Pembentukan Graph

Setelah Board terbentuk, tahap kedua adalah membangun graph sederhana. Pada graph ini, node hanya dibuat untuk posisi yang dapat menjadi tempat aktor berhenti, sedangkan edge menyatakan satu gerakan sliding lengkap. Pembentukan dilakukan dengan BFS bermula dari posisi Z. Untuk setiap node, fungsi `slide` dipanggil dengan empat arah gerak. Hasil sliding yang valid menghasilkan node tujuan baru (atau menggunakan node yang sudah ada apabila posisinya sama) beserta edge baru yang menyimpan arah, cost total, dan daftar tile penting yang dilewati. Setelah seluruh node selesai dieksplorasi, fungsi `setPredictionCosts` dipanggil untuk menghitung nilai jarak Manhattan dari setiap node menuju goal dan menuju setiap angka penting. Nilai-nilai ini akan dipakai sebagai heuristic pada GBFS dan A*.

```

1 FUNCTION BuildGraph(board) -> graph
2 ALGORITMA
3   root <- node pada posisi start Z
4   frontier.push(root)
5   processed <- empty set

```

```

6
7 WHILE frontier tidak kosong DO
8   current <- frontier.pop()
9   IF current sudah ada di processed THEN
10    continue
11   processed.insert(current)
12
13   FOR setiap arah gerak DO
14    slideResult <- Slide(board, current, arah)
15    IF slideResult tidak valid THEN
16     continue
17
18    nextNode <- node pada posisi berhenti slideResult
19    IF nextNode belum ada THEN
20     tambahkan nextNode ke graph
21     frontier.push(nextNode)
22
23    tambahkan edge current -> nextNode
24    simpan arah, cost, dan passedImportant pada edge
25
26 hitung nilai heuristic setiap node
27 RETURN graph

```

Listing 3: Pseudocode Pembentukan Graph Sederhana

2.1.3 Tahap 3: Pencarian Solusi

Setelah graph terbentuk, pencarian dilakukan dengan state yang tidak hanya menyimpan posisi node, tetapi juga angka berikutnya yang harus dilewati. Hal ini dibutuhkan karena dua jalur dapat berhenti pada node yang sama tetapi memiliki progres angka yang berbeda.

State pencarian didefinisikan sebagai pasangan (*nodeId*, *nextNumber*). Variabel *nodeId* menyatakan posisi berhenti aktor pada graph, sedangkan *nextNumber* menyatakan indeks angka penting berikutnya yang masih harus dilewati. Dengan struktur ini, dua kunjungan terhadap node yang sama dengan progres angka yang berbeda diperlakukan sebagai dua state yang berbeda. Tabel *bestCost*[(*nodeId*, *nextNumber*)] digunakan untuk mencatat cost terbaik yang pernah ditemukan pada setiap state, sehingga state dengan cost lebih buruk tidak ikut diperluas. Ekspansi suatu state dilakukan dengan menelusuri seluruh edge yang keluar dari *nodeId*. Untuk setiap edge, daftar *passedImportant* dievaluasi secara berurutan. Bila edge melewati angka yang lebih besar dari *nextNumber*, edge tersebut dianggap tidak valid karena melanggar urutan. Bila edge melewati angka yang sama atau lebih kecil, *nextNumber* diperbarui sehingga state berikutnya menyimpan progres yang sudah diperhitungkan. Disini, iterasi pada program direpresentasikan sebagai melakukannya satu kali sliding atau berpindahnya node yang ada sebanyak satu kali.

```

1 FUNCTION Search(graph, algorithm, heuristicMode) -> solution
2 ALGORITMA
3   rootState <- (rootNode, nextNumber = 0, g = 0)
4   frontier.push(rootState)
5   bestCost[rootNode, 0] <- 0
6
7   WHILE frontier tidak kosong DO
8     current <- frontier.pop dengan prioritas terbaik
9
10    IF current berada di 0 AND semua angka selesai THEN
11     RETURN path dari start ke current
12
13    FOR setiap edge dari current.node DO
14     nextNumber <- current.nextNumber
15     valid <- ApplyPassedImportant(edge.passedImportant, nextNumber)
16     IF NOT valid THEN

```

```

17         continue
18
19         newCost <- current.g + edge.cost
20         IF newCost lebih baik dari bestCost[edge.to, nextNumber] THEN
21             hitung priority sesuai UCS, GBFS, atau A*
22             frontier.push(state baru)
23
24     RETURN tidak ada solusi

```

Listing 4: Pseudocode Pathfinding

Pada implementasi ini, ketiga algoritma tidak ditulis sebagai tiga fungsi yang terpisah. Ketiganya berjalan pada satu fungsi internal `search` dengan struktur kerja yang sama: state (`nodeId`, `nextNumber`), tabel `bestCost` untuk pruning, satu `priority_queue` dengan field `priority`, pengecekan goal saat pop, validasi urutan angka pada setiap edge, serta rekonstruksi path melalui rantai `HistoryRecord`. Yang membedakan UCS, GBFS, dan A* hanya cara mengisi field `priority` dan apakah $h(n)$ perlu dihitung. Pemilihan rumus `priority` ditangani oleh sebuah lambda `priorityFor(gCost, heuristicCost)` yang mengembalikan nilai berbeda sesuai parameter `algorithm`. Berikut penjelasan langsung bagaimana setiap algoritma bekerja di dalam fungsi yang sama.

Pada UCS, parameter `algorithm` bernilai `Algorithm::UCS` sehingga `priorityFor` mengembalikan `gCost` saja. Fungsi `search` juga sengaja melewati pemanggilan `getHeuristicCost` dan langsung mengisi `heuristicCost = 0` karena heuristic tidak diperlukan. Akibatnya, state diambil dari priority queue selalu dalam urutan menaik berdasarkan cost aktual dari Z.

Pada GBFS, parameter `algorithm` bernilai `Algorithm::GBFS` sehingga `priorityFor` mengembalikan `heuristicCost` saja. Nilai `heuristicCost` diisi oleh `Graph::getHeuristicCost(node, nextNumber, mode)` sesuai mode H1, H2, atau H3 yang dipilih pengguna. Karena `gCost` tidak ikut diperhitungkan, GBFS akan mengambil state yang terlihat paling dekat dengan target dari priority queue, walaupun cost untuk sampai ke state tersebut sudah besar. Hal ini membuat GBFS sering kali cepat menemukan solusi pertama, namun solusi tersebut belum tentu memiliki cost terkecil karena ekspansi tidak mempertimbangkan jarak yang sudah ditempuh.

Pada A*, parameter `algorithm` bernilai `Algorithm::AStar` sehingga `priorityFor` mengembalikan `gCost + heuristicCost`, yaitu $f(n) = g(n) + h(n)$. Sama seperti GBFS, nilai `heuristicCost` diisi melalui `Graph::getHeuristicCost` sesuai mode H1, H2, atau H3.

Rekonstruksi path dilakukan setelah goal ditemukan dengan menelusuri rantai `HistoryRecord` dari state goal ke state awal secara mundur, kemudian dibalik agar menghasilkan urutan langkah dari awal ke akhir.

2.2 Heuristic

Untuk GBFS dan A*, program menyediakan tiga mode heuristic. Ketiga mode bekerja pada level node graph, bukan pada tile mentah, dan seluruhnya menggunakan jarak Manhattan sebagai dasar perhitungan. Dengan node n berada di posisi (r_n, c_n) , jarak Manhattan ke target di posisi (r_t, c_t) didefinisikan sebagai

$$d(n, t) = |r_n - r_t| + |c_n - c_t|.$$

H1 FinishOnly. Heuristic H1 hanya melihat jarak Manhattan dari node saat ini menuju tile finish 0. Secara matematis, $h_1(n) = d(n, O)$. Heuristic ini paling sederhana dan tidak peduli pada angka penting yang belum dilewati. Karena hanya memperhatikan finish, H1 berperilaku baik pada peta tanpa angka penting atau pada saat seluruh angka sudah dilewati. Pada peta dengan banyak angka, H1 dapat menyesatkan karena pencarian akan terdorong ke arah finish meskipun aktor masih harus berputar mengambil angka. Pada implementasi, nilai h_1 sudah dihitung saat pembentukan graph dan disimpan sebagai `finishPredictionCost` pada setiap node.

H2 ImportantSequence. Heuristic H2 mengarahkan pencarian ke angka berikutnya yang harus dilewati. Diberikan state $(node, nextNumber)$ dengan jumlah angka penting K , heuristic dihitung sebagai

$$h_2(n) = \begin{cases} d(n, T_{nextNumber}) & \text{jika } nextNumber < K, \\ d(n, O) & \text{jika } nextNumber = K, \end{cases}$$

dengan T_i adalah posisi angka penting i . Selama angka belum habis, H2 fokus mempersempit jarak ke angka berikutnya, sehingga lebih informatif daripada H1 di tahap awal pencarian. Setelah seluruh angka dilewati, H2 otomatis berperilaku seperti H1, yaitu menarik aktor ke finish. Karena hanya menjumlah satu jarak Manhattan, H2 cenderung menjadi nilai estimasi yang relatif kecil, sehingga lebih aman terhadap pelanggaran admissibility dibanding H3.

H3 OrderedSequence. Heuristic H3 mencoba memperkirakan total perjalanan yang masih tersisa dengan menjumlahkan jarak Manhattan ke seluruh target yang belum dilewati secara berurutan, lalu ke finish. Diberikan state $(node, nextNumber)$ dengan posisi $T_{nextNumber}, T_{nextNumber+1}, \dots, T_K$ sebagai rangkaian target yang masih harus dilewati, heuristic dihitung sebagai

$$h_3(n) = d(n, T_{nextNumber}) + \sum_{i=nextNumber}^{K-2} d(T_i, T_{i+1}) + d(T_{K-1}, O).$$

Apabila seluruh angka sudah selesai, h_3 langsung sama dengan $d(n, O)$. H3 paling informatif karena memperhitungkan beban perjalanan dari posisi sekarang ke seluruh target dan kemudian ke finish. Pengaruhnya jelas terlihat pada GBFS, di mana H3 sering memilih jalur yang mempertimbangkan urutan angka dengan lebih baik dibanding H1 atau H2. Namun, karena nilai estimasi H3 dapat menjadi cukup besar, H3 lebih rentan kehilangan sifat admissibility, sehingga A* dengan H3 tidak selalu menjamin solusi optimal walaupun pada banyak peta tetap mendekati optimal.

3 Analisis Algoritma

3.1 Definisi State, $g(n)$, dan $f(n)$

Ruang state pada Ice Sliding Puzzle tidak cukup hanya direpresentasikan sebagai posisi aktor. Karena ada aturan urutan tile penting yang harus dipatuhi, dua situasi berbeda dapat terjadi pada posisi papan yang sama: satu sudah mengambil checkpoint tertentu dan satu lagi belum. Kedua situasi ini harus diperlakukan berbeda oleh algoritma karena transisi yang diperbolehkan pun berbeda.

Oleh karena itu, state pencarian didefinisikan sebagai pasangan:

$$s = (nodeId, nextNumber),$$

dengan $nodeId$ menyatakan indeks node pada graph sederhana (posisi berhenti aktor) dan $nextNumber$ menyatakan indeks angka penting berikutnya yang wajib dilewati ($0 \leq nextNumber \leq K$ dengan K adalah total tile angka pada peta). State awal adalah $(root, 0)$ dan state goal adalah semua state (n, K) di mana n adalah node bertipe 0.

Nilai $g(n)$ didefinisikan sebagai total cost aktual dari state awal menuju state n , yaitu jumlah cost seluruh tile yang dilalui pada setiap gerakan sliding sepanjang jalur. Karena satu edge mewakili satu sliding penuh yang dapat melewati banyak tile, cost sebuah edge dihitung sebagai akumulasi cost seluruh tile yang dilintasi, bukan hanya tile tujuan. Secara rekursif:

$$g(n') = g(n) + c(n, n'),$$

dengan $c(n, n')$ adalah cost edge dari n ke n' . Oleh karena itu, $g(n) \geq 0$ selalu terpenuhi karena cost setiap tile dijamin tidak negatif berdasarkan validasi input.

Pada A*, fungsi evaluasi $f(n)$ menggabungkan cost aktual dengan estimasi sisa perjalanan:

$$f(n) = g(n) + h(n).$$

Pada UCS, $h(n) = 0$ untuk setiap state sehingga $f(n) = g(n)$. Pada GBFS, komponen $g(n)$ diabaikan dan prioritas ditetapkan murni oleh $h(n)$. Definisi $h(n)$ untuk masing-masing mode H1, H2, dan H3 telah dijelaskan pada Subbab 2.2.

Untuk menghindari ekspansi berulang pada state yang sama dengan cost lebih tinggi, program menyimpan tabel `bestCost[(nodeId, nextNumber)]`. Setiap kali akan memasukkan state baru ke frontier, program memeriksa apakah state tersebut sudah pernah ditemukan dengan cost yang lebih murah. Jika ya, state baru tersebut dibuang langsung tanpa dimasukkan ke frontier. Mekanisme ini secara efektif melakukan *pruning* terhadap jalur-jalur yang sudah pasti tidak optimal, sehingga jumlah state yang benar-benar diekspansi jauh lebih kecil dari jumlah state yang pernah ditemukan.

3.2 Admissibility dan Konsistensi Heuristic

Suatu heuristic $h(n)$ disebut *admissible* apabila untuk setiap state n berlaku $h(n) \leq h^*(n)$, dengan $h^*(n)$ adalah cost optimal sebenarnya dari state n menuju goal. Sifat ini menjamin bahwa A* tidak pernah meremehkan jarak yang sebenarnya, sehingga ketika goal pertama kali di-pop dari frontier, cost yang ditemukan sudah pasti optimal. Kondisi yang lebih kuat adalah *konsistensi* (atau monotonicity), yaitu $h(n) \leq c(n, n') + h(n')$ untuk setiap transisi ke n' melalui edge berbiaya $c(n, n')$. Heuristic konsisten selalu admissible, tetapi kebalikannya tidak selalu berlaku. Pada A* dengan heuristic konsisten, setiap state yang di-pop dari frontier dijamin sudah memiliki cost optimal sehingga tidak perlu diproses ulang.

Pada Ice Sliding Puzzle, ketiga heuristic H1, H2, dan H3 tidak dapat dijamin admissible maupun konsisten untuk semua kemungkinan peta. Penyebab pertama bersumber dari sifat gerakan sliding itu sendiri: karena satu gerakan dapat melewati banyak tile sekaligus, jarak Manhattan antara dua node bisa jauh lebih kecil dari cost aktual apabila lintasan melewati tile-tile berbiaya tinggi, tetapi juga bisa lebih besar dari cost aktual apabila slide panjang melewati banyak tile berbiaya rendah. Dalam kasus kedua, heuristic melebihi-lebihkan dan kehilangan sifat admissibility-nya. Penyebab kedua lebih spesifik pada H3, yang menjumlahkan jarak Manhattan ke seluruh target yang tersisa secara berurutan. Penjumlahan ini mengasumsikan setiap segmen dapat ditempuh secara independen tanpa kendala lintasan sliding, padahal pada kenyataannya transisi antar checkpoint dapat memerlukan gerakan memutar yang jauh lebih mahal dari pergerakan Manhattan, atau justru lebih murah bila slide tunggal menempuh banyak tile sekaligus. Akibatnya H3 lebih rentan menghasilkan overestimate yang besar dibanding H1 maupun H2.

Meskipun tidak ada jaminan admissibility secara teoritis, ketiga heuristic tetap berguna sebagai panduan arah pencarian. Pada sebagian besar peta dengan distribusi cost yang tidak ekstrem, estimasi Manhattan cukup mendekati cost aktual sehingga A* masih menemukan solusi yang optimal atau mendekati optimal, dan GBFS dapat menemukan solusi dengan jumlah iterasi yang jauh lebih sedikit dibanding UCS.

3.3 Perbandingan Sifat Formal Algoritma

Ketiga algoritma merupakan algoritma yang sama dengan hanya berbeda pada cara mengisi nilai prioritas di frontier. Pada A*, prioritas setiap state dihitung sebagai $f(n) = g(n) + h(n)$. UCS dapat dipandang sebagai kasus khusus A* dengan $h(n) = 0$ untuk seluruh state sehingga $f(n) = g(n)$, sedangkan GBFS merupakan kasus khusus lainnya di mana komponen $g(n)$ diabaikan sepenuhnya sehingga $f(n) = h(n)$.

Tahap pembentukan graph bergantung pada ukuran papan. Misalkan papan berukuran $R \times C$ sehingga jumlah tile maksimum adalah $R \cdot C$. Jumlah node graph V paling banyak sama dengan jumlah tile yang dapat menjadi posisi berhenti. Untuk setiap node, program

mencoba empat arah sliding dan menelusuri tile-tile hingga berhenti, dengan panjang slide maksimum $L = \max(R, C)$, sehingga kompleksitas pembentukan graph adalah $O(V \cdot L)$. Setelah graph terbentuk, fungsi `setPredictionCosts` menghitung jarak Manhattan dari setiap node ke setiap target, titik finish dan seluruh angka penting. Dengan K angka penting dan V node, kompleksitas tahap ini adalah $O(V \cdot K)$, sehingga total kompleksitas pembentukan graph menjadi $O(VL + VK)$.

Tahap pencarian beroperasi pada ruang state berukuran $|S| = V \cdot (K + 1)$, karena setiap posisi node dapat dikombinasikan dengan $K + 1$ kemungkinan nilai `nextNumber`. Setiap state dimasukkan ke frontier paling banyak satu kali karena tabel `bestCost` memblokir pemrosesan ulang state dengan cost yang sama atau lebih besar. Operasi push dan pop pada `priority_queue` membutuhkan $O(\log |S|)$. Jumlah edge yang perlu dievaluasi paling banyak $E \cdot (K + 1)$ karena setiap edge dapat diproses pada setiap nilai `nextNumber`. Dengan demikian, kompleksitas waktu pencarian adalah:

$$T_{\text{pencarian}} = O(E \cdot (K + 1) \cdot \log(V \cdot (K + 1))) = O(EK \log(VK)).$$

Pada peta tanpa tile penting ($K = 0$), rumus ini tereduksi menjadi $O(E \log V)$, yaitu kompleksitas standar Dijkstra atau A*. Dari sisi ruang, implementasi ini membutuhkan tabel `bestCost` berukuran $O(VK)$, vektor `history` yang menyimpan satu rekaman per state ($O(VK)$), serta frontier yang pada kondisi terburuk menyimpan seluruh state ($O(VK)$), sehingga total kompleksitas ruang adalah $O(VK)$.

Tahap	Kompleksitas Waktu	Kompleksitas Ruang
Pembentukan graph	$O(VL + VK)$	$O(V + E)$
Pencarian (UCS/A*/GBFS)	$O(EK \log(VK))$	$O(VK)$
Total	$O(VL + EK \log(VK))$	$O(VK + E)$

Perbedaan di antara ketiganya bukan terletak pada batas terburuk, melainkan pada berapa banyak state yang *benar-benar* diekspansi sebelum goal ditemukan yang secara langsung ditentukan oleh formula prioritas masing-masing.

Karena komponen $g(n)$ ikut diperhitungkan pada A*, algoritma ini tidak hanya mempertimbangkan estimasi jarak ke target seperti halnya GBFS, tetapi juga akumulasi cost yang sudah ditempuh sejak posisi awal. Akibatnya, A* lebih tahan terhadap heuristic yang kurang akurat: apabila $h(n)$ sedikit menyesatkan, komponen $g(n)$ masih berperan sebagai penyeimbang yang mencegah pencarian terus menjelajahi jalur-jalur yang sudah terbukti mahal. GBFS tidak memiliki mekanisme penyeimbang ini sehingga rentan terjebak pada jalur yang secara heuristic tampak menjanjikan, namun akumulasi costnya jauh lebih besar dibanding jalur alternatif yang secara heuristic kurang menonjol. Perbedaan cara mengisi $f(n)$ inilah yang pada akhirnya menentukan sifat kelengkapan dan optimalitas masing-masing algoritma.

Algoritma	Ada solusi? (Complete?)	Optimalitas	Kompleksitas Waktu
UCS	Ada	Optimal ($c \geq 0$)	$O(EK \log(VK))$
GBFS	Ada [†]	Tidak optimal	$O(EK \log(VK))$
A*	Ada	Optimal (jika h admissible)	$O(EK \log(VK))$

[†]GBFS memiliki solusi pada graph berhingga dengan mekanisme pruning `bestCost`.

Ketiga algoritma bersifat *complete* pada implementasi ini karena graph yang dibentuk berhingga dan tabel `bestCost` memastikan tidak ada state yang diekspansi lebih dari satu kali dengan cost yang sama atau lebih buruk, sehingga siklus tak hingga tidak dapat terjadi. Tanpa mekanisme

ini, GBFS berpotensi terjebak dalam siklus apabila heuristic secara berulang mengarahkan pencarian kembali ke state yang sebelumnya sudah dikunjungi.

Optimalitas UCS - Ketika sebuah state di-pop, nilai $g(n)$ yang dimilikinya merupakan yang terkecil di antara seluruh state yang masih berada di frontier maupun yang belum ditemukan. Pada kondisi awal, hanya state awal yang berada di frontier dengan $g = 0$, yang secara trivial benar. Setelah setiap operasi pop, seluruh state yang baru dimasukkan ke frontier memiliki $g \geq g(\text{current})$ karena cost setiap edge dijamin tidak negatif. Dengan demikian, ketika state goal pertama kali di-pop, tidak ada jalur lebih murah yang belum dieksplorasi, sehingga cost yang ditemukan sudah pasti merupakan minimum.

Optimalitas A* - Optimalitas A* bergantung pada sifat admissibility heuristic yang digunakan. Misalkan cost jalur optimal ke goal adalah C^* . Apabila $h(n) \leq h^*(n)$ untuk setiap state n , maka ketika goal di-pop dengan cost $g(\text{goal})$, seluruh state n' yang masih di frontier memiliki $f(n') \leq g(n') + h^*(n')$. Jika terdapat jalur optimal yang belum dieksplorasi, pasti ada state n' pada jalur tersebut dengan $f(n') \leq C^*$. Apabila $g(\text{goal}) > C^*$, maka $f(n') \leq C^* < g(\text{goal})$, artinya n' seharusnya di-pop sebelum goal. Oleh karena itu disimpulkan $g(\text{goal}) = C^*$. Pada implementasi ini, admissibility tidak selalu terjamin karena heuristic berbasis Manhattan dapat melebih-lebihkan cost pada peta sliding (lihat Subbab 3.2), sehingga optimalitas A* tidak dapat dijamin secara umum.

Optimalitas GBFS - Berbeda dengan UCS dan A*, GBFS tidak memiliki jaminan optimalitas karena komponen $g(n)$ diabaikan sepenuhnya. Frontier GBFS selalu mengeluarkan state dengan $h(n)$ terkecil tanpa mempertimbangkan cost yang sudah dikeluarkan untuk mencapai state tersebut.

Dalam praktik, UCS mengekspansi state berdasarkan urutan $g(n)$ tanpa panduan arah sehingga pencarian menyebar ke semua arah secara merata dan menghasilkan jumlah ekspansi yang paling banyak. GBFS mengekspansi state yang terlihat paling dekat ke target sehingga lebih terarah, tetapi bila heuristic menyesatkan akibat banyaknya rintangan, GBFS justru membuang iterasi dalam jumlah besar. A* menyeimbangkan kedua kecenderungan tersebut: $h(n)$ mengarahkan ekspansi ke arah yang menjanjikan, sementara $g(n)$ memastikan jalur yang sudah terbukti mahal tidak terus diekspansi. Apabila heuristic kurang informatif, perilaku A* secara bertahap mendekati UCS karena $g(n)$ semakin mendominasi nilai $f(n)$.

4 Implementasi

Program dikembangkan menggunakan bahasa pemrograman **C++17**. Program terdiri atas versi CLI dan GUI. Versi CLI dibangun menggunakan **Makefile**, sedangkan versi GUI dibangun menggunakan **CMake** dan **raylib**. Struktur utama program dibagi ke dalam beberapa file, yaitu **Board**, **Graph**, **PathFinding**, **I0**, **main**, dan **mainGUI**.

4.1 Source Program

Source program ditulis menggunakan bahasa C++17 dan dibagi ke dalam folder **include** dan **src**. Folder **include** berisi deklarasi struktur data dan class, sedangkan folder **src** berisi implementasi. Pembagian file source program adalah sebagai berikut.

File	Keterangan
include/Board.hpp dan src/Board.cpp	Menyimpan representasi board, validasi input, cost tile, posisi start/finish, dan simulasi sliding.
include/graph.hpp dan src/graph.cpp	Menyimpan representasi graph sederhana, node, edge, heuristic per node, dan proses pembentukan graph dari board.

File	Keterangan
include/pathFinding.hpp dan src/pathFinding.cpp	Menyimpan implementasi UCS, GBFS, dan A* serta validasi urutan angka saat edge dilewati.
include/IO.hpp dan src/IO.cpp	Menyimpan fungsi input/output bersama untuk CLI dan GUI, termasuk path input, path output, dan penyimpanan solusi.
src/main.cpp	Entry point CLI untuk menerima input pengguna, menjalankan algoritma, dan menampilkan hasil.
src/mainGUI.cpp	Entry point GUI berbasis raylib untuk input file, pemilihan algoritma, visualisasi board, playback path, dan penyimpanan solusi.

4.2 Struktur Data

Struktur data utama program berada pada `Board.hpp` dan `graph.hpp`. Struktur `Tile` menyimpan informasi posisi, cost, dan tipe tile. Struktur `Position` menyimpan koordinat baris dan kolom. Struktur `SlideResult` menyimpan hasil satu gerakan sliding.

```

1 struct Tile {
2     int x = 0;
3     int y = 0;
4     int cost = 0;
5     char type = '*';
6 };
7
8 struct Position {
9     int row = 0;
10    int col = 0;
11 };
12
13 struct PassedTile {
14     int row = 0;
15     int col = 0;
16     char type = '*';
17 };
18
19 struct SlideResult {
20     bool valid = false;
21     int stopRow = 0;
22     int stopCol = 0;
23     int cost = 0;
24     char direction = '?';
25     std::vector<PassedTile> passedImportant;
26 };

```

Listing 5: Struktur Data Board

Graph dibentuk menggunakan pointer antar node. Setiap `GraphNode` menyimpan daftar `Edge`. Setiap edge mengarah langsung ke node tetangga, sehingga pathfinding dapat mengakses node tujuan tanpa perlu mencari ulang berdasarkan id.

```

1 struct Edge {
2     GraphNode* neighbor = nullptr;
3     int neighborId = -1;
4     int costEdge = 0;
5     int costPrediction = 0;
6     char direction = '?';
7     bool canStopAtNeighbor = true;
8     std::vector<PassedTile> passedImportant;
9 };

```

```

10
11 struct GraphNode {
12     int nodeId = -1;
13     int row = 0;
14     int col = 0;
15     char type = '*';
16     int finishPredictionCost = 0;
17     std::vector<PredictionCost> predictionCosts;
18     std::vector<Edge> neighbors;
19 };

```

Listing 6: Struktur Data Graph

4.3 Pembacaan Input

Pembacaan input dilakukan oleh fungsi `readBoardFromFile`. Fungsi ini membaca ukuran papan, baris peta, dan matriks cost. Program juga melakukan validasi agar input sesuai format. Beberapa kondisi yang dianggap salah adalah tidak adanya start Z, tidak adanya finish 0, jumlah Z atau 0 lebih dari satu, baris kosong, panjang baris tidak sesuai, cost kurang atau lebih, serta adanya data tambahan setelah matriks cost.

```

1 if (startCount != 1) {
2     throw std::runtime_error("Peta harus memiliki tepat satu titik awal Z.");
3 }
4 if (goalCount != 1) {
5     throw std::runtime_error("Peta harus memiliki tepat satu titik tujuan 0.");
6 }
7
8 for (int number = 0; number <= maxNumber; ++number) {
9     if (!foundNumbers[number]) {
10         throw std::runtime_error("Angka pada peta harus berurutan mulai dari 0."
11         );
12     }
13 }

```

Listing 7: Validasi Input Board

Input file dapat dipanggil langsung dari path penuh, path relatif, atau nama file saja. Apabila pengguna hanya memasukkan nama file seperti `test.txt`, program akan mencarinya pada folder `test/input/test.txt`. Pengaturan ini dibuat pada modul IO.

4.4 Simulasi Sliding

Gerakan aktor disimulasikan oleh fungsi `slide`. Fungsi ini menerima posisi awal, delta arah, dan karakter arah. Selama bergerak, fungsi akan menjumlahkan cost tile yang dilewati dan menyimpan angka atau finish yang dilewati ke dalam `passedImportant`. Jika gerakan keluar papan, melewati lava, atau langsung menabrak rintangan tanpa bergerak, hasil sliding dianggap tidak valid.

```

1 while (true) {
2     int nextRow = currentRow + deltaRow;
3     int nextCol = currentCol + deltaCol;
4
5     if (!isInside(nextRow, nextCol)) {
6         return SlideResult{};
7     }
8
9     if (isWall(nextRow, nextCol)) {
10         if (!hasMoved) {
11             return SlideResult{};
12         }
13     }
14 }

```

```

13
14     result.valid = true;
15     result.stopRow = currentRow;
16     result.stopCol = currentCol;
17     return result;
18 }
19
20 if (isLava(nextRow, nextCol)) {
21     return SlideResult{};
22 }
23 }

```

Listing 8: Inti Simulasi Sliding

4.5 Pembuatan Graph

Graph dibuat dari papan dengan memulai dari posisi Z. Program melakukan BFS terhadap node yang dapat menjadi posisi berhenti. Untuk setiap node, program mencoba empat arah sliding. Jika hasil sliding valid, program membuat atau mengambil node tujuan, lalu menambahkan edge dari node saat ini ke node tujuan.

```

1 for (int i = 0; i < 4; ++i) {
2     SlideResult result = board.slide(currentNode->row, currentNode->col,
3                                     DELTA_ROW[i], DELTA_COL[i], DIRECTIONS[i]);
4     if (!result.valid) {
5         continue;
6     }
7
8     GraphNode* nextNode = getOrAddNode(result.stopRow, result.stopCol,
9                                       board.getType(result.stopRow, result.
10 stopCol));
11
12     addNeighbor(currentNode, nextNode, result.cost,
13                 board.getHeuristicCost(result.stopRow, result.stopCol),
14                 result.direction, result.passedImportant);
15 }

```

Listing 9: Pembentukan Edge Graph

4.6 Pencarian Solusi

Pencarian solusi dilakukan pada kelas `PathFinding`. Algoritma UCS, GBFS, dan A* memakai fungsi internal yang sama, yaitu `search`. Perbedaannya berada pada perhitungan prioritas.

```

1 const auto priorityFor = [algorithm](int gCost, int heuristicCost) {
2     if (algorithm == Algorithm::UCS) {
3         return gCost;
4     }
5     if (algorithm == Algorithm::GBFS) {
6         return heuristicCost;
7     }
8     return gCost + heuristicCost;
9 };

```

Listing 10: Prioritas Pencarian

Sebelum state baru dimasukkan ke frontier, edge yang dilewati akan divalidasi terhadap urutan angka. Jika edge melewati angka yang lebih besar dari angka yang seharusnya, edge tersebut tidak digunakan.

```

1 bool advanceImportantSequence(int currentNext,
2                               const std::vector<PassedTile>& passedTiles,

```

```

3         int& nextAfter) {
4     nextAfter = currentNext;
5     for (const PassedTile& tile : passedTiles) {
6         if (tile.type < '0' || tile.type > '9') {
7             continue;
8         }
9
10        int digit = tile.type - '0';
11        if (digit < nextAfter) {
12            continue;
13        }
14        if (digit == nextAfter) {
15            ++nextAfter;
16            continue;
17        }
18        return false;
19    }
20    return true;
21 }

```

Listing 11: Validasi Urutan Angka pada Edge

4.7 Antarmuka CLI dan GUI

Program memiliki dua antarmuka. Antarmuka CLI berada pada `main.cpp`. Pengguna dapat memasukkan file input, memilih algoritma, memilih heuristic untuk GBFS atau A*, dan memilih apakah solusi akan disimpan.

Antarmuka GUI berada pada `mainGUI.cpp`. GUI dibuat menggunakan `raylib`. Pada GUI, pengguna dapat mengetik nama file input, memilih algoritma, memilih heuristic, menjalankan solver, melihat visualisasi board, melakukan playback langkah solusi, dan menyimpan hasil solusi.

4.8 Modul IO

Modul IO dibuat agar CLI dan GUI tidak memiliki kode input/output yang berulang. Fungsi di dalam modul ini digunakan untuk menyelesaikan path input, membentuk path output, mengubah path solusi menjadi string, mencetak board dengan posisi aktor, serta menyimpan solusi ke file.

```

1 std::string resolveInputPath(const std::string& path);
2 std::string getOutputPath(const std::string& inputPath);
3 std::string directionsToString(const std::vector<char>& path);
4
5 void saveSolution(const Board& board,
6                 const std::vector<char>& path,
7                 const std::vector<Position>& positions,
8                 int totalCost,
9                 int totalIterations,
10                long long elapsedMs,
11                const std::string& outputPath);

```

Listing 12: Fungsi IO Bersama

4.9 Penyimpanan Hasil

Hasil solusi dapat disimpan dalam bentuk file teks pada folder `test/output`. File output berisi informasi path, total cost, jumlah iterasi, waktu eksekusi, serta visualisasi posisi aktor pada setiap langkah solusi.

5 Pengujian

5.1 Kasus Uji 1: test1.txt – UCS

```
7 7
XXXXXX
XO****X
X**X**X
X****OX
X1***LX
XZ**X*X
XXXXXX
999 999 999 999 999 999 999
999 3 5 2 8 1 999
999 7 4 999 6 9 999
999 2 8 3 5 4 999
999 6 1 7 2 999 999
999 9 3 4 999 8 999
999 999 999 999 999 999 999
```

Input: test1.txt

```
Solusi Yang Ditemukan : RULUDRUR
Cost dari Solusi : 87
Initial
XXXXXX
XO****X
X**X**X
X****OX
X1***LX
XZ**X*X
XXXXXX
...
>> Waktu eksekusi: 0.014 ms
>> Banyak iterasi yang dilakukan: 13
    iterasi
```

Output

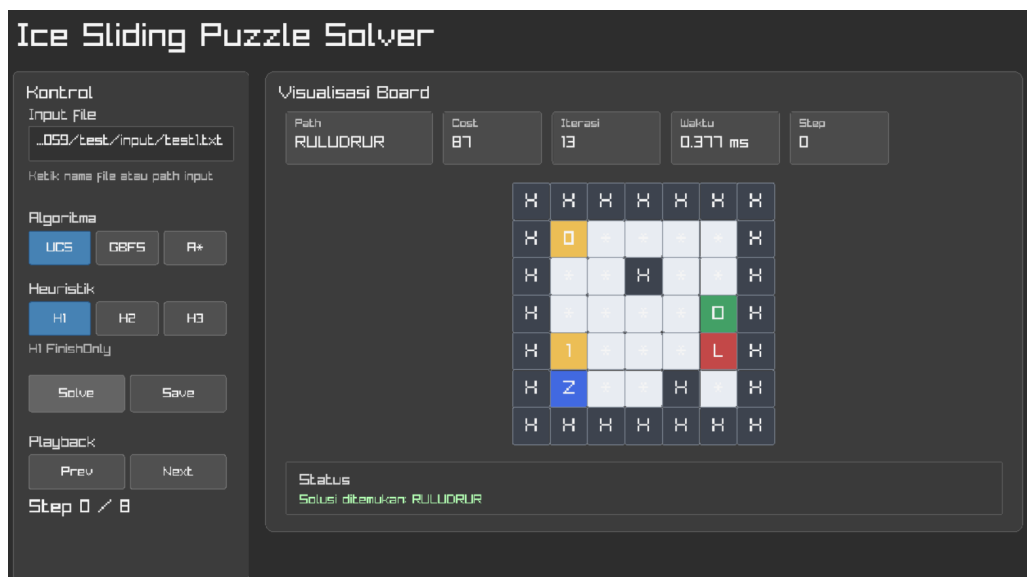


Figure 1: Tampilan GUI kasus uji 1 (test1.txt, UCS).

5.2 Kasus Uji 2: test2.txt – GBFS H1

```

13 17
XXXXXXXXXXXXXXXXX
X***X*****X
XZ**0***X0***2X
X**1***X*X**6XX
XX*****X*X***X
X*****X*****X
X*****X*****X
X4***8***X***X
X*7*****5XX
X*****XX*****X
X*****9*****X
X*3*****X
XXXXXXXXXXXXXXXXX
999 999 999 999 999 999 999 999 999 999 999 999 999 999
999 999
999 9 5 1 6 999 7 3 8 4 9 5 1 6 2 7 999
999 3 8 4 9 5 1 6 2 999 3 8 4 9 5 1 999
999 6 2 7 3 8 4 9 5 999 6 999 7 3 8 999 999
999 999 5 1 6 2 7 3 8 999 9 999 1 6 2 7 999
999 3 8 4 9 5 1 6 2 999 3 8 4 9 5 1 999
999 6 2 7 3 8 4 9 5 1 6 2 7 3 8 4 999
999 9 5 1 6 2 7 3 8 4 9 999 1 6 2 7 999
999 3 8 4 9 5 1 6 2 7 3 8 4 9 5 999 999
999 6 2 7 3 8 4 9 999 999 6 2 7 3 8 4 999
999 9 5 1 6 2 7 3 8 4 9 5 1 6 2 7 999
999 3 8 4 9 5 1 6 2 7 3 8 4 9 5 1 999
999 999 999 999 999 999 999 999 999 999 999 999 999
999 999

```

Input: test2.txt

```

Solusi Yang Ditemukan :
RLDRURDLDLURDRUDL
URDLURULDRUDL
Cost dari Solusi : 924
Initial
XXXXXXXXXXXXXXXXX
X***X*****X
XZ**0***X0***2X
X**1***X*X**6XX
XX*****X*X***X
X*****X*****X
X*****X*****X
X4***8***X***X
X*7*****5XX
X*****XX*****X
X*****9*****X
X*3*****X
XXXXXXXXXXXXXXXXX
...
>> Waktu eksekusi: 0.083 ms
>> Banyak iterasi yang dilakukan: 113
iterasi

```

Output

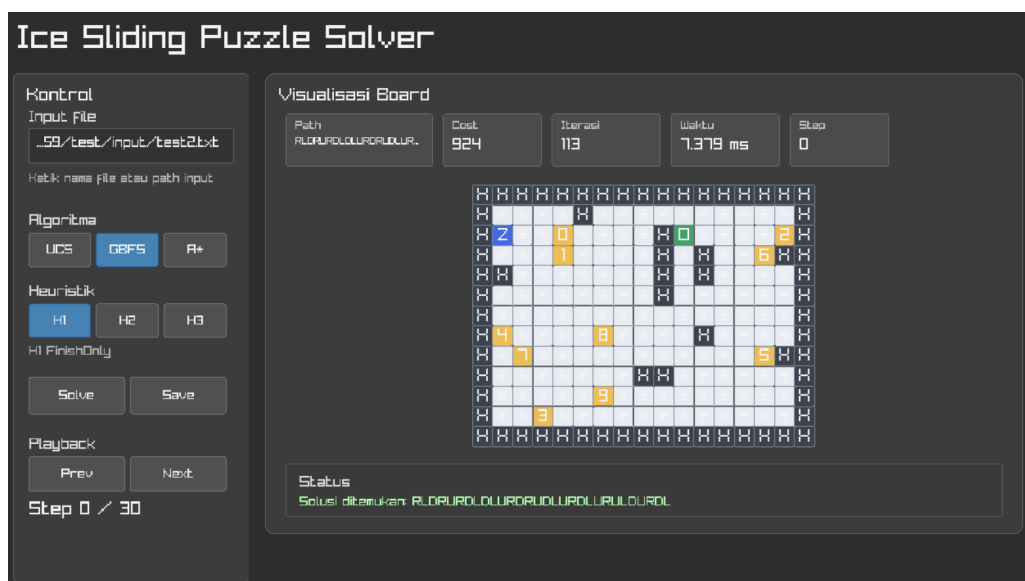


Figure 2: Tampilan GUI kasus uji 2 (test2.txt, GBFS H1).

5.3 Kasus Uji 3: test2.txt – GBFS H2

```

13 17
XXXXXXXXXXXXXXXXXX
X***X*****X
XZ**0***X0***2X
X**1***X*X**6XX
XX*****X*X***X
X*****X*****X
X*****X*****X
X4***8***X***X
X*7*****5XX
X*****XX*****X
X*****9*****X
X*3*****X***X
XXXXXXXXXXXXXXXXXX
999 999 999 999 999 999 999 999 999 999 999 999 999 999
    999 999
999 9 5 1 6 999 7 3 8 4 9 5 1 6 2 7 999
999 3 8 4 9 5 1 6 2 999 3 8 4 9 5 1 999
999 6 2 7 3 8 4 9 5 999 6 999 7 3 8 999 999
999 999 5 1 6 2 7 3 8 999 9 999 1 6 2 7 999
999 3 8 4 9 5 1 6 2 999 3 8 4 9 5 1 999
999 6 2 7 3 8 4 9 5 1 6 2 7 3 8 4 999
999 9 5 1 6 2 7 3 8 4 9 999 1 6 2 7 999
999 3 8 4 9 5 1 6 2 7 3 8 4 9 5 999 999
999 6 2 7 3 8 4 9 999 999 6 2 7 3 8 4 999
999 9 5 1 6 2 7 3 8 4 9 5 1 6 2 7 999
999 3 8 4 9 5 1 6 2 7 3 8 4 9 5 1 999
999 999 999 999 999 999 999 999 999 999 999 999 999
    999 999

```

Input: test2.txt

```

Solusi Yang Ditemukan :
RLDRURDLDLURDRUDL
URDLURULDRUDL
Cost dari Solusi : 924
Initial
XXXXXXXXXXXXXXXXXX
X***X*****X
XZ**0***X0***2X
X**1***X*X**6XX
XX*****X*X***X
X*****X*****X
X*****X*****X
X4***8***X***X
X*7*****5XX
X*****XX*****X
X*****9*****X
X*3*****X***X
XXXXXXXXXXXXXXXXXX
...
>> Waktu eksekusi: 0.066 ms
>> Banyak iterasi yang dilakukan: 102
    iterasi

```

Output

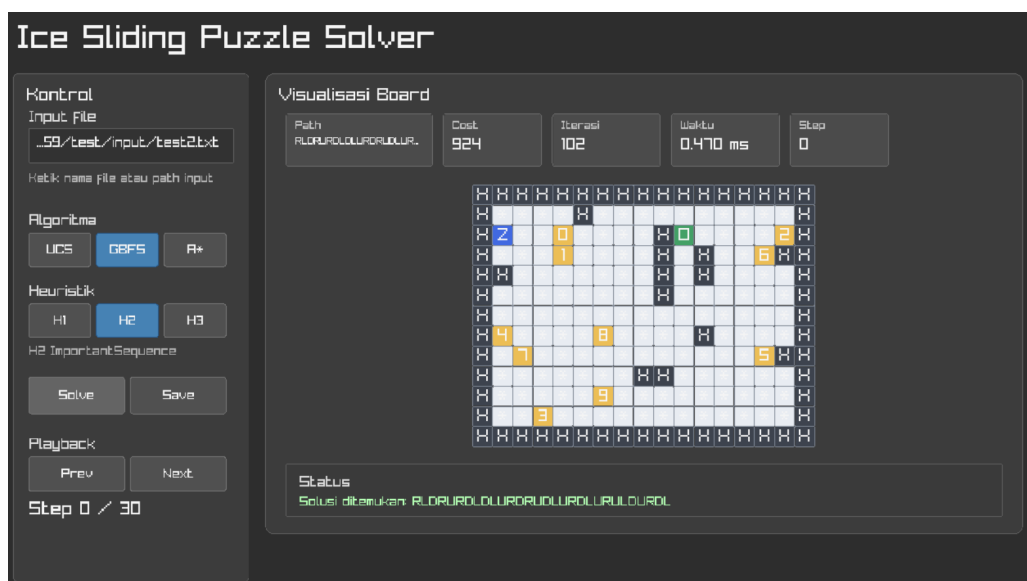


Figure 3: Tampilan GUI kasus uji 3 (test2.txt, GBFS H2).

```

8 26
XXXXXXXXXXXXXXXXXXXXXXXXX
X*****X
X**X*****X**X
X*****X
X**4***X*****X
XX*****X
X*****X**X
X*****X**5X
X*****XX9XX*****XX
X*****ZX0*****X
X1**XX***XX8XX***XX**X
X*****3X**X*****X
X**X*****X
XX*****XX
X2*****7*****6X
X*****X*****X
X*****X*****X
XXXXXXXXXXXXXXXXXXXXXXXXX
999 999 999 999 999 999 999 999 999 999 999 999 999 999 999 999 999 999 999
    999 999 999 999 999 999 999 999 999 999 999 999 999 999
999 9 5 1 6 2 7 3 8 4 9 5 1 6 2 7 3 8 4 9 5 1 6 2 7 999
999 3 8 4 999 5 1 6 2 7 3 8 4 9 5 1 6 2 7 999 8 4 9 5 1 999
999 6 2 7 3 8 4 9 5 1 6 2 7 3 8 4 9 5 1 6 2 7 3 8 4 999
999 9 5 1 6 2 7 3 999 4 9 5 1 6 2 7 3 8 4 9 5 1 6 2 7 999
999 999 999 4 9 5 1 6 2 7 3 8 4 9 5 1 6 2 7 3 8 4 9 5 1 6 2 7 999
999 6 2 7 3 8 4 9 5 1 6 2 7 3 8 4 9 5 1 6 2 7 3 8 4 999
999 9 5 1 6 2 7 3 8 4 9 999 1 999 2 7 3 8 4 9 999 1 6 2 7 999
999 3 8 4 9 5 1 6 2 7 999 999 4 999 999 1 6 2 7 3 8 4 9 999
    999 999
999 6 2 7 3 8 4 9 5 1 6 999 7 999 8 4 9 5 1 6 2 7 3 8 4 999
999 9 5 1 6 999 999 3 8 4 999 999 1 999 999 7 3 8 4 999 999 1
    6 2 7 999
999 3 8 4 9 5 1 6 2 7 3 999 4 999 5 1 6 2 7 3 8 4 9 5 1 6 999
999 6 2 7 999 8 4 9 5 1 6 2 7 3 8 4 9 5 1 6 2 7 3 8 4 999
999 999 999 1 6 2 7 3 8 4 9 5 1 6 2 7 3 8 4 9 5 1 6 999 999
    999
999 3 8 4 9 5 1 6 2 7 3 8 4 9 5 1 6 2 7 3 8 4 9 5 1 999
999 6 2 7 3 8 4 999 5 1 6 2 7 3 8 4 9 5 1 6 2 7 3 8 4 999
999 9 5 1 6 2 7 3 8 4 9 999 1 6 2 7 3 8 4 9 5 1 6 2 7 999
999 999 999 999 999 999 999 999 999 999 999 999 999 999 999
    999 999 999 999 999 999 999 999 999 999

```

Output

[illegible]

18

5.5 Kasus Uji 5: test3.txt – A* H3

```
18 26
XXXXXXXXXXXXXXXXXXXXXXXXX
X*****X
X**X*****X***X
X*****X
X*4***X*****X
XXX*****X
X*****X***X
X*****X*X*****X*5X
X*****XX9XX*****XX
X*****ZX0*****0*****X
X1**XX***XX8XX***XX***X
X*****3X*X*****X
X**X*****X
XXX*****XXX
X2*****7*****6X
X*****X*****X
X*****X*****X
XXXXXXXXXXXXXXXXXXXXXXXXX
999 999 999 999 999 999 999 999 999 999 999 999 999 999 999
    999 999 999 999 999 999 999 999 999 999
999 9 5 1 6 2 7 3 8 4 9 5 1 6 2 7 3 8 4 9 5 1 6 2 7 999
999 3 8 4 999 5 1 6 2 7 3 8 4 9 5 1 6 2 7 999 8 4 9 5 1 999
999 6 2 7 3 8 4 9 5 1 6 2 7 3 8 4 9 5 1 6 2 7 3 8 4 999
999 9 5 1 6 2 7 3 999 4 9 5 1 6 2 7 3 8 4 9 5 1 6 2 7 999
999 999 999 4 9 5 1 6 2 7 3 8 4 9 5 1 6 2 7 3 8 4 9 5 1 999
999 6 2 7 3 8 4 9 5 1 6 2 7 3 8 4 9 5 1 6 999 7 3 8 4 999
999 9 5 1 6 2 7 3 8 4 9 999 1 999 2 7 3 8 4 9 999 1 6 2 7 999
999 3 8 4 9 5 1 6 2 7 999 999 4 999 999 1 6 2 7 3 8 4 9 999
    999 999
999 6 2 7 3 8 4 9 5 1 6 999 7 999 8 4 9 5 1 6 2 7 3 8 4 999
999 9 5 1 6 999 999 3 8 4 999 999 1 999 999 7 3 8 4 999 999 1
    6 2 7 999
999 3 8 4 9 5 1 6 2 7 3 999 4 999 5 1 6 2 7 3 8 4 9 5 1 999
999 6 2 7 999 8 4 9 5 1 6 2 7 3 8 4 9 5 1 6 2 7 3 8 4 999
999 999 999 1 6 2 7 3 8 4 9 5 1 6 2 7 3 8 4 9 5 1 6 999 999
    999
999 3 8 4 9 5 1 6 2 7 3 8 4 9 5 1 6 2 7 3 8 4 9 5 1 999
999 6 2 7 3 8 4 999 5 1 6 2 7 3 8 4 9 5 1 6 2 7 3 8 4 999
999 9 5 1 6 2 7 3 8 4 9 999 1 6 2 7 3 8 4 9 5 1 6 2 7 999
999 999 999 999 999 999 999 999 999 999 999 999 999 999 999
    999 999 999 999 999 999 999 999 999 999 999 999
```

Input: test3.txt

```
Solusi Yang Ditemukan :
LURDRDLULUDRDLUDR
ULDRURDLDRUDLULDR
DLDRULURDL
Cost dari Solusi : 2452
Initial
XXXXXXXXXXXXXXXXXXXXXXXXX
X*****X
X**X*****X***X
X*****X
X*4***X*****X
XXX*****X
X*****X***X
X*****X*X*****X*5X
X*****XX9XX*****XX
X*****ZX0*****0*****X
X1**XX***XX8XX***XX***X
X*****3X*X*****X
X**X*****X
XXX*****XXX
X2*****7*****6X
X*****X*****X
X*****X*****X
XXXXXXXXXXXXXXXXXXXXXXXXX
...
>> Waktu eksekusi: 0.122 ms
>> Banyak iterasi yang dilakukan: 217
    iterasi
```

Output

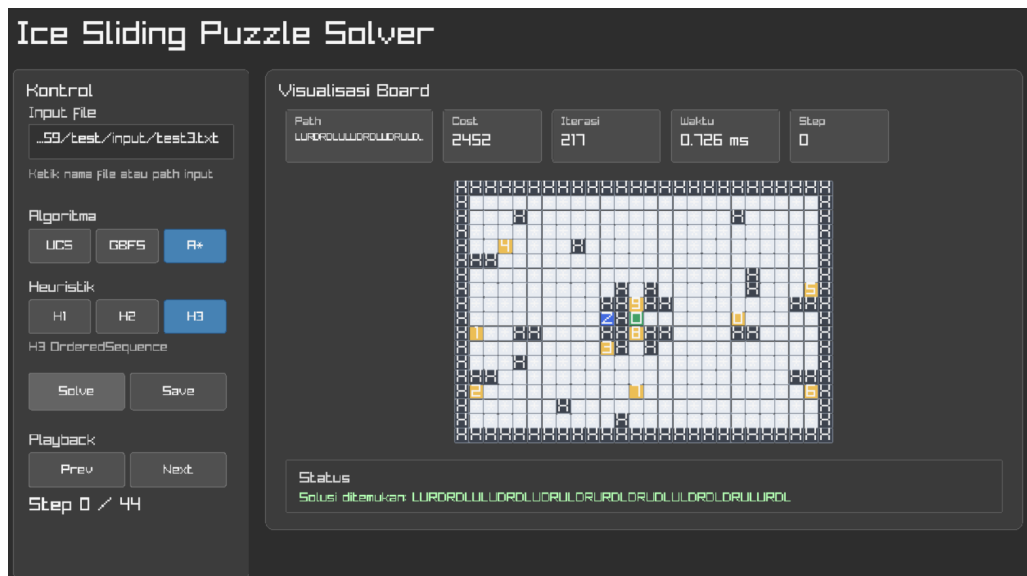


Figure 5: Tampilan GUI kasus uji 5 (test3.txt, A* H3).

5.6 Analisis Hasil Percobaan

Tabel berikut merangkum hasil seluruh kasus uji yang telah dilakukan.

Uji	File	Algoritma	Path	Cost	Iterasi
1	test1.txt	UCS	RULUDRUR	87	13
2	test2.txt	GBFS H1	RLDRURDLURDRUDL URDLURULDRUDL	924	113
3	test2.txt	GBFS H2	RLDRURDLURDRUDL URDLURULDRUDL	924	102
4	test3.txt	A* H1	LURDRDLULURDLUDR ULDRURDLURDLULDR DLDRULURDL	2452	217
5	test3.txt	A* H3	LURDRDLULURDLUDR ULDRURDLURDLULDR DLDRULURDL	2452	217

6 Kesimpulan dan Saran

6.1 Kesimpulan

Permainan Ice Sliding Puzzle dapat dimodelkan sebagai persoalan pencarian jalur pada graph sederhana, di mana setiap node mewakili posisi berhenti aktor dan setiap edge mewakili satu gerakan sliding lengkap beserta cost akumulatif dan daftar tile penting yang dilewati. Pendekatan ini memungkinkan representasi yang ringkas karena hanya posisi-posisi yang benar-benar dapat menjadi tempat berhenti yang dijadikan node, bukan seluruh tile pada papan.

Aturan urutan tile angka ditangani dengan memperluas definisi state pencarian menjadi pasangan (*nodeId*, *nextNumber*). Dengan demikian, dua kunjungan ke posisi yang sama dengan progres angka yang berbeda diperlakukan sebagai dua state yang berbeda dan tidak saling menggugurkan. Mekanisme tabel **bestCost** berperan sebagai pruning untuk mencegah ekspansi ulang state dengan cost yang lebih buruk.

Ketiga algoritma UCS, GBFS, dan A* dapat diimplementasikan dalam satu fungsi pencarian yang sama, dengan perbedaan hanya pada formula prioritas. UCS menggunakan $g(n)$ saja sehingga menjamin solusi optimal selama cost edge tidak negatif. GBFS menggunakan $h(n)$ saja sehingga lebih cepat menemukan solusi tetapi tidak ada jaminan optimalitas. A* menggabungkan keduanya dengan $f(n) = g(n) + h(n)$, menjamin optimalitas apabila heuristic yang digunakan bersifat admissible.

Heuristic berbasis jarak Manhattan tidak selalu admissible pada permainan sliding karena satu gerakan dapat melewati banyak tile sekaligus dengan cost kumulatif yang berbeda-beda dari estimasi Manhattan. Meskipun demikian, pada sebagian besar peta uji ketiga heuristic H1, H2, dan H3 tetap memberikan panduan arah pencarian yang efektif sehingga A* maupun GBFS dapat menemukan solusi yang optimal atau mendekati optimal dalam jumlah iterasi yang jauh lebih sedikit dibanding UCS.

6.2 Saran

Beberapa saran pengembangan program di masa mendatang adalah sebagai berikut.

1. Heuristic dapat dikembangkan agar lebih informatif dan tetap admissible, misalnya dengan mempertimbangkan struktur rintangan dan cost minimum tile pada peta.
2. Visualisasi GUI dapat diperluas dengan animasi pergerakan aktor dan penanda jalur yang sudah ditelusuri agar pengguna lebih mudah memahami solusi.
3. Dukungan format input dapat diperluas, misalnya dengan menambahkan validasi yang lebih informatif terhadap kesalahan format file.

4. Algoritma pencarian dapat diperluas dengan varian seperti IDA* atau Bidirectional Search untuk dibandingkan performanya pada peta berukuran lebih besar.

7 Daftar Pustaka

1. Munir, R. (2026). *Route Planning (Bagian 1)*. Program Studi Teknik Informatika, Sekolah Teknik Elektro dan Informatika, Institut Teknologi Bandung. [https://informatika.stei.itb.ac.id/~rinaldi.munir/Stmik/2025-2026/21-Route-Planning-\(2026\)-Bagian1.pdf](https://informatika.stei.itb.ac.id/~rinaldi.munir/Stmik/2025-2026/21-Route-Planning-(2026)-Bagian1.pdf).
2. Munir, R. (2026). *Route Planning (Bagian 2)*. Program Studi Teknik Informatika, Sekolah Teknik Elektro dan Informatika, Institut Teknologi Bandung. [https://informatika.stei.itb.ac.id/~rinaldi.munir/Stmik/2025-2026/22-Route-Planning-\(2026\)-Bagian2.pdf](https://informatika.stei.itb.ac.id/~rinaldi.munir/Stmik/2025-2026/22-Route-Planning-(2026)-Bagian2.pdf).
3. Russell, S., & Norvig, P. (2020). *Artificial Intelligence: A Modern Approach* (4th ed.). Pearson.
4. Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to Algorithms* (4th ed.). MIT Press.
5. Dokumentasi resmi raylib. <https://www.raylib.com/>.

8 Lampiran

No	Poin	Ya	Tidak
1	Program berhasil di kompilasi tanpa kesalahan	✓	
2	Program berhasil dijalankan	✓	
3	Solusi yang diberikan program benar dan mematuhi aturan permainan	✓	
4	Program dapat membaca masukan berkas <code>.txt</code> serta menyimpan solusi dalam berkas <code>.txt</code>	✓	
5	Program memiliki Graphical User Interface (GUI)	✓	
6	Ada algoritma <i>pathfinding</i> alternatif selain dari spesifikasi wajib (Algoritma X dan Algoritma Y)		✓
7	Ada tambahkan dua alternatif heuristik selain yang utama (Heuristik X dan Heuristik Y)	✓	

Tugas ini disusun sepenuhnya tanpa bantuan kecerdasan buatan (Generative AI), melainkan hasil pemikiran dan analisis mandiri.



Raymond Jonathan Dwi Putra Julianto

Tautan Repositori: https://github.com/Alpaomega1136/Tucil3_13524059